

CAR HIRE MANAGEMENT PLATFORM

System Documentation — Chapters One, Four & Five

CHAPTER ONE: INTRODUCTION

1.1 Introduction

The car hire industry in Kenya, particularly within urban centres such as Nairobi, remains heavily dependent on manual and semi-manual processes for fleet management, booking coordination, and customer verification. Vehicle owners and small-to-medium car hire operators typically rely on phone calls, WhatsApp messages, physical logbooks, and spreadsheet-based tracking to manage bookings, monitor vehicle availability, and reconcile payments. This approach introduces significant inefficiencies: double-bookings occur because availability is not centrally visible, security deposits are handled informally with little audit trail, vehicle condition disputes arise from the absence of standardised inspection records, and administrators have no real-time visibility into where a rented vehicle currently is or whether it has exceeded its agreed operating zone.

This project proposes the design and development of a web-based Car Hire Management Platform built on the MERN stack (MongoDB, Express.js, React, Node.js) with TypeScript, intended to digitise and automate the end-to-end car hire lifecycle — from vehicle discovery and booking, through KYC verification and payment, to fleet monitoring and post-rental reconciliation. The system introduces features that go beyond conventional booking CRUD operations, including automated security deposit handling, digital inspection checklists, telematics-based live tracking, and business intelligence dashboards for fleet operators.

1.2 Background of the Study

The client operating in this domain is a small-to-medium car hire business based in Nairobi that maintains a mixed fleet of saloon cars, SUVs, and multi-purpose vehicles (such as the Toyota Noah and Voxy) which are hired out to individual customers and corporate clients for both within-Nairobi and upcountry trips. Currently, the business advertises available vehicles informally through social media and word of mouth, and each booking enquiry is handled manually by an administrator who checks a shared spreadsheet or physical diary to confirm vehicle availability. Security deposits are collected in cash or via mobile money without a

standardised refund workflow, and vehicle condition before and after rental is recorded, if at all, through informal photographs sent over WhatsApp with no central archive.

As the fleet has grown, this manual model has become increasingly difficult to sustain. The business has no consolidated view of fleet utilisation, no automated way to flag overdue returns, and no mechanism to verify a customer's identity documents before releasing a vehicle. There is also no live visibility into a vehicle's location once it leaves the yard, which exposes the business to risk in cases of vehicles being taken outside agreed operating zones or not returned on time. The proposed system is intended to replace these manual workflows with a centralised, automated digital platform.

1.3 Problem Statement

The following problems, drawn directly from the client's current mode of operation, motivate this project:

1. Lack of centralised availability visibility: Because bookings are tracked on spreadsheets and diaries updated by different staff, double-bookings and scheduling conflicts occur when two customers are promised the same vehicle for overlapping dates.
2. Informal and unauditable deposit handling: Security deposits are collected and refunded manually with no automated hold-and-release mechanism, leading to disputes over deposit amounts and delayed refunds.
3. Absence of standardised vehicle condition records: Pre-rental and post-rental vehicle inspections are not consistently documented, making it difficult to resolve disputes over new scratches, dents, or missing accessories.
4. No real-time fleet or location visibility: Administrators cannot see, at a glance, which vehicles are available, rented, in maintenance, or overdue, nor can they verify whether a vehicle has left its agreed operating zone.
5. Manual, error-prone overdue and penalty calculation: Late return penalties are calculated manually and inconsistently, sometimes resulting in under-billing or customer disputes.

6. No structured KYC process: Customer identity documents (driving licences, passports) are collected informally with no expiry tracking or formal verification workflow before a vehicle is released.
7. Limited business intelligence: The business has no consolidated reporting on revenue per vehicle category, fleet utilisation, or outstanding dues, making it difficult to make informed fleet investment decisions.

1.4 Objectives

1.4.1 General Objective

To design, develop, and test a web-based Car Hire Management Platform, built on the MERN stack with TypeScript, that automates vehicle booking, customer verification, deposit management, fleet tracking, and reporting for a Nairobi-based car hire business.

1.4.2 Specific Objectives

8. To investigate and document the current manual car hire booking and fleet management processes used by the client, including data gathering on booking, deposit, and inspection workflows, within the first two weeks of the project.
9. To design a relational and document-based data model (conceptual, logical, and physical) capable of supporting dynamic vehicle inventory, bookings, KYC records, and inspection checklists.
10. To develop a customer-facing booking portal in React and TypeScript that supports search and filtering by vehicle type, seating capacity, and transmission, side-by-side vehicle comparison, and destination-based dynamic pricing, completed within the development phase of the project.
11. To develop an administrative command centre with a real-time, colour-coded fleet status matrix, a digital inspection checklist module, and a KYC verification queue.
12. To implement an automated security deposit workflow that pre-authorises and releases refundable holds, integrated with regional payment gateways such as M-Pesa and Stripe.
13. To implement a live vehicle tracking module with geo-fencing alerts that notify administrators when a vehicle marked for within-Nairobi use crosses out of the designated zone.

14. To implement an automated notification and overdue-penalty engine that sends SMS/email alerts via Twilio or SendGrid and calculates late fees hourly once a grace period elapses.
15. To test and evaluate the developed system against the defined functional and non-functional requirements using unit, integration, and user acceptance testing prior to project submission.

1.5 Scope and Limitation of the Study

The scope of this project covers the design and development of the core car hire platform as outlined in the feature blueprint: the customer-facing portal (search and discovery, booking and add-ons, payments and KYC), the admin command centre (fleet and telematics management, customer and booking operations, business intelligence and reporting), and shared system automation (notifications and overdue penalty calculation). The system will be developed as a web application using React with TypeScript on the frontend, Node.js and Express.js on the backend, and MongoDB as the primary data store, with Socket.io used for real-time updates such as live fleet status and tracking.

The following are explicitly outside the scope of this project: a native mobile application (the system will be a responsive web application only); integration with physical GPS hardware, which will instead be simulated for demonstration purposes given cost and time constraints, with the architecture designed to accommodate real hardware integration in future work; and full production-grade integration with live payment gateway merchant accounts, which will instead be implemented and tested against sandbox/test environments of the chosen payment providers.

1.6 Justification

This project is justified on several grounds. First, it addresses a real, currently unmet need identified directly from the client's day-to-day operational pain points, meaning the resulting system has an immediate practical beneficiary rather than being a purely academic exercise. Second, the project is technically interesting and appropriately challenging for a final-year undergraduate project: it requires the integration of several non-trivial subsystems, including real-time communication (Socket.io), third-party payment and messaging APIs, geolocation/geo-fencing logic, and document-based data modelling, which together demonstrate competence across the full stack.

Third, the timing is favourable: mobile money penetration and API-driven payment infrastructure in Kenya (M-Pesa, Flutterwave) have matured to a point where such integrations are realistic within a student project timeline, and the broader shift toward digitised SME operations makes this a timely contribution. Finally, successful realisation of the project would give the client a competitive, professional booking experience comparable to larger international car hire platforms, while giving the developer a portfolio-grade demonstration of applied MERN and TypeScript engineering aligned with AIoT-adjacent features such as live telematics.

1.7 Project Risks and Mitigation

Risk	Mitigation Strategy
Scope creep from the extensive feature blueprint (telematics, KYC, deposits, analytics all being pursued at once).	Adopt an iterative delivery plan: implement and stabilise core booking and fleet management first, then layer in telematics, deposits, and analytics as incremental milestones.
Dependence on third-party APIs (M-Pesa, Flutterwave, Stripe, Twilio, SendGrid, Google Maps/Mapbox) that may have sign-up delays, sandbox limitations, or rate limits.	Register for developer/sandbox accounts early in the project timeline; build the payment and notification modules behind an internal abstraction layer so a provider can be swapped with minimal code change.
Limited access to real GPS hardware for telematics testing.	Simulate GPS coordinate streams via a scheduled backend service during development and demonstration, with the data model designed to accept real hardware payloads unchanged.
Data privacy risk from storing KYC documents (driving licences, passports).	Store documents in an access-controlled document vault, encrypt sensitive fields at rest, and restrict access to authenticated admin roles only.
Time constraints given the academic project schedule versus the breadth of the feature blueprint.	Prioritise objectives using MoSCoW (Must/Should/Could/Won't) classification and track progress against the Gantt chart in Section 1.9.
Single-developer bottleneck (no team redundancy).	Maintain version control discipline (feature branching, regular commits) so partially completed features do not block a working demo build at any milestone.

1.8 Budget and Resources

The project is primarily resource-light in terms of hardware, relying instead on existing developer equipment and free or student-tier cloud services. The table below summarises the anticipated resources.

Resource Category	Details
Hardware	Existing personal development laptop; a Redmi Android device used for responsive/mobile testing.
Software / Tooling	Visual Studio Code, Node.js/npm, MongoDB (local + Atlas free tier), Git/GitHub, Figma and UX Pilot for interface design, Postman for API testing.
Cloud & Hosting	Free/student tiers of a frontend host (e.g. Vercel), a backend host, and MongoDB Atlas for the database.
Third-Party APIs	Sandbox/developer accounts: M-Pesa Daraja API, Stripe test mode, Twilio and/or SendGrid trial tiers, Google Maps or Mapbox free-tier API keys.
Human Resources	Project developer (researcher), academic supervisor for guidance and review, client/business owner as a domain informant for requirements validation.
Other Costs	Internet connectivity for development and API testing; incidental costs for printing and binding of project documentation.

1.9 Project Schedule

The project is planned across the standard academic project timeline, broken down into requirement gathering and analysis, system and database design, iterative development (core booking, then payments/KYC, then telematics/analytics), testing, and documentation/write-up phases. The detailed Work Breakdown Structure, Gantt chart, and network diagram showing the critical path are provided in the appendix section.

[INSERT DIAGRAM: Project Gantt Chart]

Insert Gantt chart (task list vs. timeline, weeks 1–14) generated from the Work Breakdown Structure, exported from your planning tool (e.g. Excel, GanttProject, or MS Project) and placed in the Appendix as referenced here.

[INSERT DIAGRAM: Project Network Diagram (Critical Path)]

Insert the activity-on-node network diagram showing task dependencies and the highlighted critical path, to be placed in the Appendix as referenced here.

CHAPTER FOUR: SYSTEM ANALYSIS AND REQUIREMENT MODELING

4.1 Introduction

This chapter presents an analysis of the client's current, largely manual car hire operation and translates the findings into formal requirement definitions and models for the proposed system. It begins with a description of how the current system operates, the fact-finding methods used to gather this information, and modelling of the current process using flowcharts, data flow diagrams (DFDs), and use case diagrams. It then presents the functional and non-functional requirements of the proposed platform and models these requirements using use case diagrams and descriptions, forming the analytical foundation for the system design presented in Chapter Five.

4.2 Description of the Current System

The client's current car hire operation is manual and communication-driven. A prospective customer typically contacts the business by phone, WhatsApp, or social media direct message to enquire about vehicle availability for a given date range. An administrator manually checks a shared spreadsheet or physical diary to determine whether a suitable vehicle (matching the requested type, seating capacity, and transmission) is free for those dates. If available, the administrator verbally quotes a rate that depends on whether the trip is within Nairobi or upcountry, and the booking is recorded by hand in the diary or spreadsheet.

Before releasing the vehicle, the administrator requests a copy of the customer's driving licence and national ID, typically sent as a photograph over WhatsApp, and collects a security deposit in cash or via mobile money with no formal receipt beyond an SMS confirmation. A walkaround inspection of the vehicle is sometimes performed and photographed, but there is no standard checklist or central storage location for these images. Once the customer returns the vehicle, the administrator visually compares its condition to memory or scattered photographs, calculates any late fee manually if the vehicle was returned after the agreed time, and refunds the deposit (minus any deductions) in cash or via mobile money transfer.

Fleet oversight is similarly manual: the business owner periodically calls each driver or checks in physically to determine whether a vehicle is on the road, at the yard, or in for maintenance. There is no live location tracking, and geo-fencing (restricting a vehicle to an agreed operating zone) is not enforced beyond a verbal agreement with the customer. Reporting on revenue and

vehicle utilisation, where it happens at all, is compiled manually and infrequently from the spreadsheet records.

4.2.1 Current System Process Flow

The diagram below documents the current, manual end-to-end process from customer enquiry through vehicle return, as elicited during requirements gathering with the client.

[INSERT DIAGRAM: Flowchart of the Current Manual Car Hire Process]

Insert a flowchart showing: Customer Enquiry → Manual Availability Check (diary/spreadsheet) → Verbal Quote → Manual Booking Record → Manual KYC Collection (WhatsApp) → Cash/M-Pesa Deposit → Ad-hoc Vehicle Inspection → Vehicle Release → Manual Fleet Monitoring → Vehicle Return → Manual Condition Comparison → Manual Late-Fee Calculation → Manual Deposit Refund.

4.3 Fact-Finding Methods and Data Gathered

A combination of fact-finding techniques was used to gather information about the current system and to elicit requirements for the proposed platform:

16. Interviews: Semi-structured interviews were conducted with the business owner/administrator to understand the booking workflow, deposit handling practices, KYC collection, and pain points around overdue vehicles and fleet visibility.
17. Direct Observation: The researcher observed a sample of live booking enquiries being handled by the administrator, including how spreadsheet records were updated and how vehicle condition was assessed on handover and return.
18. Document Review: Existing artefacts — the booking spreadsheet, sample WhatsApp KYC exchanges, and mobile money transaction records — were reviewed (with sensitive data anonymised) to understand the data currently being captured and its gaps.
19. Questionnaires: A short questionnaire was administered to a sample of past customers to understand their experience of the booking, deposit, and vehicle handover process, and to gauge appetite for online payment and self-service booking.

Key findings from this fact-finding exercise directly informed the problem statement in Chapter One and the functional requirements defined in Section 4.5: absence of a single source of truth for vehicle availability, no audit trail for deposits, inconsistent KYC and inspection records, and no live fleet or location visibility.

4.4 Requirement Definition and Modelling of the Current System

The current system's data flow is modelled below using a Data Flow Diagram (DFD) to show how information (booking requests, KYC documents, payments) moves between the customer, the administrator, and the manual records (spreadsheet/diary) that serve as the current system's only 'store'. A use case diagram is also provided to model the current, manual interactions between the customer and administrator actors.

[INSERT DIAGRAM: Context-Level and Level-1 DFD of the Current Manual System]

Insert a Context Diagram (Level 0) showing Customer and Administrator as external entities interacting with the 'Manual Car Hire Process', and a Level-1 DFD decomposing this into: 1.0 Handle Enquiry, 2.0 Record Booking, 3.0 Collect KYC & Deposit, 4.0 Inspect & Release Vehicle, 5.0 Process Return & Refund — each reading/writing the manual 'Booking Diary/Spreadsheet' data store.

[INSERT DIAGRAM: Use Case Diagram – Current Manual System]

Insert a use case diagram with actors Customer and Administrator, showing use cases: Enquire About Vehicle, Record Booking Manually, Collect KYC Documents, Collect Deposit, Inspect Vehicle, Release Vehicle, Process Return, Calculate Late Fee Manually, Refund Deposit.

4.5 Requirement Definitions and Specifications of the Proposed System

Drawing on the feature blueprint developed for the platform and the gaps identified in the current system, this section specifies the functional and non-functional requirements of the proposed Car Hire Management Platform, organised by the three major subsystems: the Customer-Facing Portal, the Admin Command Centre, and Shared System Automation.

4.5.1 Functional Requirements — Customer-Facing Portal

Req. ID	Functional Requirement
FR-01	The system shall allow customers to search and filter available vehicles by date range, vehicle type, seating capacity, engine capacity (CC), and transmission type.
FR-02	The system shall display an availability timeline on a vehicle's detail page indicating the next date it becomes free if currently booked.
FR-03	The system shall allow customers to select up to three vehicles for a side-by-side comparison of price, CC, and luggage capacity.
FR-04	The system shall provide a destination toggle (Within Nairobi / Outside Nairobi) that instantly recalculates the displayed rental rate.

Req. ID	Functional Requirement
FR-05	The system shall allow customers to add optional services to a booking, including a chauffeur, child/baby car seats, and extended insurance (excess waiver).
FR-06	The system shall allow a customer to request a booking extension from their active dashboard before the scheduled return time.
FR-07	The system shall charge the rental fee and place a separate, refundable, pre-authorised security deposit hold at checkout.
FR-08	The system shall integrate with regional payment providers (M-Pesa, Flutterwave, or Stripe) to process rental and deposit payments.
FR-09	The system shall provide a document vault allowing customers to securely upload driving licences and passports, with expiry-date tracking.

4.5.2 Functional Requirements — Admin Command Centre

Req. ID	Functional Requirement
FR-10	The system shall display a real-time fleet matrix showing each vehicle's status as Available, Rented, In Maintenance, or Overdue Return, colour-coded accordingly.
FR-11	The system shall display active rentals on an interactive map using simulated or real GPS coordinates via a mapping API (Google Maps or Mapbox).
FR-12	The system shall raise a geo-fencing alert to the administrator when a vehicle designated Within Nairobi crosses outside the defined boundary.
FR-13	The system shall maintain a digital maintenance logbook recording odometer readings, service history, and upcoming scheduled maintenance per vehicle.
FR-14	The system shall provide a KYC verification queue allowing an administrator to approve or reject customer-submitted identity documents.
FR-15	The system shall display a live rental manifest listing active rentals with customer name, phone number, destination, and emergency contact.
FR-16	The system shall allow administrators to record a digital inspection checklist, including uploaded photographs, at vehicle release and return.
FR-17	The system shall generate financial analytics showing gross revenue and revenue per vehicle category, and outstanding dues.

Req. ID	Functional Requirement
FR-18	The system shall generate utilisation reports showing rental frequency per vehicle over a selected period.

4.5.3 Functional Requirements — Shared System Automation

Req. ID	Functional Requirement
FR-19	The system shall automatically send SMS and email notifications (via Twilio/SendGrid) for booking confirmation, upcoming return reminders, and overdue alerts.
FR-20	The system shall automatically calculate late-return penalty fees on an hourly basis once a booking exceeds its grace period.

4.5.4 Non-Functional Requirements

Req. ID	Non-Functional Requirement
NFR-01	Performance: Core booking pages shall load within 2–3 seconds under normal network conditions.
NFR-02	Scalability: The backend (Node.js/Express) and database (MongoDB) shall be structured to accommodate growth in fleet size and concurrent bookings without redesign.
NFR-03	Security: KYC documents and payment tokens shall be encrypted at rest and in transit; role-based access control shall restrict admin functions to authenticated staff accounts.
NFR-04	Reliability: Real-time features (fleet matrix, tracking, notifications) shall degrade gracefully (e.g. fall back to last-known state) if a Socket.io connection is temporarily lost.
NFR-05	Usability: The customer portal shall be responsive and usable on common mobile screen sizes, given the target market's high mobile usage.
NFR-06	Maintainability: The codebase shall use TypeScript across frontend and backend to enforce type safety and ease future maintenance.
NFR-07	Auditability: All deposit holds, releases, and deductions shall be logged with timestamps to support dispute resolution.

4.5.5 Use Case Modelling of the Proposed System

The proposed system introduces three primary actors — Customer, Administrator, and the system's own Automation Engine (a system actor responsible for scheduled/triggered processes such as overdue calculation and notifications) — together with an external Payment Gateway actor. The use case diagram below models the major interactions of the proposed system as derived from the functional requirements in Sections 4.5.1–4.5.3.

[INSERT DIAGRAM: Use Case Diagram – Proposed Car Hire Management Platform]

Insert a use case diagram with actors: Customer, Administrator, Payment Gateway (external), Automation Engine (system). Include use cases such as: Search & Filter Vehicles, Compare Vehicles, Book Vehicle, Add Optional Services, Request Extension, Make Payment «include» Place Deposit Hold, Upload KYC Documents, View Fleet Matrix, Track Vehicle Location, Manage Geo-fence Alerts, Verify KYC, Record Inspection Checklist, View Financial Analytics, Send Notification «extend» from Booking Confirmed / Return Due / Overdue, Calculate Late Fee.

4.5.6 Selected Use Case Descriptions

Field	UC-01: Book Vehicle with Deposit
Actor(s)	Customer, Payment Gateway
Precondition	Customer has selected an available vehicle for a valid date range and is logged in with a verified account.
Main Flow	1. Customer selects destination (Within/Outside Nairobi) and optional add-ons. 2. System calculates total rental fee and required deposit amount. 3. Customer confirms and is redirected to the payment gateway. 4. Payment gateway processes rental payment and places a pre-authorised deposit hold. 5. System creates the booking record and marks the vehicle as Rented from the start date.
Postcondition	Booking is confirmed; confirmation notification is sent to the customer; vehicle status updates on the admin fleet matrix.
Exceptions	Payment declined → booking is not created and customer is prompted to retry; vehicle becomes unavailable between selection and payment → customer is notified and offered alternatives.

Field	UC-02: Geo-fence Breach Alert
Actor(s)	Automation Engine, Administrator
Precondition	A vehicle is currently rented and marked Within Nairobi; live location updates are being received.
Main Flow	1. System receives a GPS coordinate update for the vehicle. 2. System checks the coordinate against the Nairobi boundary polygon. 3. If the coordinate falls outside the boundary, system raises a Geo-fence Alert. 4. Alert is pushed to the admin dashboard in real time via Socket.io and logged.
Postcondition	Administrator is notified in real time and can view the vehicle's current position on the map.
Exceptions	GPS signal lost → system flags the vehicle as 'location unknown' rather than raising a false breach alert.

CHAPTER FIVE: SYSTEM DESIGN

5.1 Introduction

This chapter translates the requirements defined in Chapter Four into a concrete system design. It presents the overall system architecture, the conceptual, logical, and physical database design, the design of key inputs and outputs, the interface design approach, the process design for critical workflows, and the security design that protects customer and business data across the platform.

5.2 System Architecture Overview

The proposed platform follows a three-tier architecture implemented with the MERN stack and TypeScript. The presentation tier is a React and TypeScript single-page application serving both the customer portal and the admin command centre from a shared component library, differentiated by role-based routing. The application tier is a Node.js/Express.js REST API, also written in TypeScript, exposing resource-oriented endpoints for vehicles, bookings, users, payments, inspections, and analytics, alongside a Socket.io server for real-time channels (fleet matrix updates, live tracking, geo-fence alerts, and notifications). The data tier is a MongoDB database accessed through an object-document mapper, supplemented by external service integrations for payments (M-Pesa/Flutterwave/Stripe), messaging (Twilio/SendGrid), and mapping (Google Maps/Mapbox).

Cross-cutting concerns — authentication (JWT-based), authorisation (role-based access control for Customer vs Administrator), and the automation engine (a scheduled job runner responsible for overdue-fee calculation and reminder notifications) — sit within the application tier and are consumed by both the REST and real-time layers.

[INSERT DIAGRAM: High-Level System Architecture Diagram]

Insert an architecture diagram showing: [React + TypeScript Client] ↔ [Express.js REST API + Socket.io Server (Node.js/TypeScript)] ↔ [MongoDB], with side integrations to Payment Gateway (M-Pesa/Flutterwave/Stripe), Notification Services (Twilio/SendGrid), and Mapping API (Google Maps/Mapbox), and an Automation Engine (scheduled jobs) module within the API layer.

5.3 Database Design

The database design follows the standard three-level approach — conceptual, logical, and physical — to move from a technology-independent model of the business entities to the concrete MongoDB collection structures that implement them.

5.3.1 Conceptual Design

At the conceptual level, the core entities of the system are: Vehicle, Booking, Customer, Administrator, Payment, SecurityDeposit, InspectionChecklist, KYCDocument, MaintenanceRecord, and Notification. A Customer makes one or more Bookings; each Booking is associated with exactly one Vehicle, one Payment record, and one SecurityDeposit; each Booking may have up to two InspectionChecklist records (pre-rental and post-rental); a Customer submits one or more KYCDocuments; a Vehicle has many MaintenanceRecords over its lifetime; and Notifications are generated against a Booking and delivered to a Customer or Administrator.

[INSERT DIAGRAM: Entity-Relationship (Conceptual) Diagram]

Insert an ER diagram showing entities Customer, Administrator, Vehicle, Booking, Payment, SecurityDeposit, InspectionChecklist, KYCDocument, MaintenanceRecord, and Notification, with cardinalities: Customer (1:N) Booking, Vehicle (1:N) Booking, Booking (1:1) Payment, Booking (1:1) SecurityDeposit, Booking (1:N) InspectionChecklist, Customer (1:N) KYCDocument, Vehicle (1:N) MaintenanceRecord, Booking (1:N) Notification.

5.3.2 Logical Design

At the logical level, each conceptual entity is expressed as a document schema with defined attributes and relationships, independent of MongoDB-specific storage details. Given MongoDB's document model, frequently co-accessed, bounded sub-entities (such as an inspection checklist's line items, or add-on selections within a booking) are embedded within their parent document, while entities that are independently queried, grow unboundedly, or are shared across parents (such as Vehicles, Customers, and Payments) are referenced by ObjectId. The key logical entities and their attributes are summarised below.

Entity	Key Attributes
Vehicle	vehicleId, make, model, type (SUV/Saloon/Van), seatingCapacity, engineCC, transmission, dailyRateWithinNairobi, dailyRateOutsideNairobi, status, currentOdometer, geoZone.
Customer	customerId, name, phone, email, passwordHash, kycStatus, emergencyContact.

Entity	Key Attributes
Booking	bookingId, customerId (ref), vehicleId (ref), startDate, endDate, destinationType, addOns[], rentalAmount, status, extensionRequests[].
Payment	paymentId, bookingId (ref), provider, rentalAmountPaid, transactionRef, status, timestamp.
SecurityDeposit	depositId, bookingId (ref), holdAmount, status (Held/Released/Deducted), deductionReason, releasedAt.
InspectionChecklist	checklistId, bookingId (ref), stage (Pre/Post), items[] (embedded: component, condition, photoUrl), inspectedBy, timestamp.
KYCDocument	documentId, customerId (ref), documentType, fileUrl, expiryDate, verificationStatus, verifiedBy.
MaintenanceRecord	recordId, vehicleId (ref), serviceType, odometerAtService, serviceDate, nextDueDate.
Notification	notificationId, bookingId (ref), recipientType, channel (SMS/Email), triggerType, sentAt.

5.3.3 Physical Design

At the physical level, each logical entity maps to a MongoDB collection. The table below specifies the physical schema for the two highest-traffic collections — vehicles and bookings — including field types and indexing strategy; the remaining collections (payments, securitydeposits, inspectionchecklists, kydocuments, maintenancerecords, notifications, customers) follow the same pattern and should be documented in full using the same structure alongside your ER and schema diagrams.

Field (vehicles)	Type / Index
_id	ObjectId (primary key)
make, model	String
type	String, enum ['SUV','Saloon','Van'], indexed
seatingCapacity	Number, indexed
engineCC	Number
transmission	String, enum ['Manual','Automatic']
dailyRateWithinNairobi / dailyRateOutsideNairobi	Number
status	String, enum ['Available','Rented','Maintenance','Overdue'], indexed

Field (vehicles)	Type / Index
currentOdometer	Number
geoZone	GeoJSON Polygon, 2dsphere indexed

Field (bookings)	Type / Index
_id	ObjectId (primary key)
customerId	ObjectId, ref: Customer, indexed
vehicleId	ObjectId, ref: Vehicle, indexed
startDate, endDate	Date, compound-indexed with vehicleId for availability queries
destinationType	String, enum ['WithinNairobi','OutsideNairobi']
addOns	Array of embedded subdocuments {type, dailyCost}
rentalAmount	Number
status	String, enum ['Confirmed','Active','Completed','Overdue','Cancelled'], indexed
extensionRequests	Array of embedded subdocuments {requestedEndDate, status}

[INSERT DIAGRAM: Full Physical Database Schema Diagram]

Insert the complete physical schema diagram (all collections, field types, indexes, and reference relationships) generated from your modelling tool (e.g. MongoDB Compass schema view, dbdiagram.io, or Lucidchart), covering payments, securitydeposits, inspectionchecklists, kydocuments, maintenancerecords, notifications, and customers in the same level of detail as the tables above.

5.4 Input Design

Inputs are designed to minimise friction for customers while capturing all data required by the requirements in Chapter Four. Key input forms include: the Vehicle Search & Filter form (date range picker, vehicle type, seating capacity, transmission); the Booking form (destination toggle, add-on checkboxes, computed price summary); the KYC Upload form (drag-and-drop file input with client-side validation of file type and size before upload to the document vault); and the Admin Inspection Checklist form (a structured, itemised checklist with condition dropdowns per vehicle component and an attached photo upload per item, used identically at pre-rental and post-rental stages to ensure directly comparable records).

All forms apply client-side validation (required fields, date-range sanity checks, file-type/size limits) mirrored by server-side validation in the Express API to ensure data integrity regardless of client behaviour.

5.5 Output Design

Key system outputs include: booking confirmation and reminder messages (SMS/email, generated from templated content merged with booking data); the real-time Fleet Matrix view (a colour-coded grid, generated from live vehicle status queries); the Financial Analytics dashboard (charts for gross revenue, revenue per vehicle category, and outstanding dues, generated from aggregation pipelines over the payments and bookings collections); the Utilisation Report (a ranked table/chart of rental frequency per vehicle over a selectable date range); and the Digital Inspection Report (a side-by-side pre-/post-rental comparison view generated from the two InspectionChecklist records tied to a booking, used to justify any deposit deduction).

5.6 Interface Design

The interface design follows a consistent design system across the customer portal and admin command centre, to be produced in Figma/UX Pilot prior to implementation. The customer portal prioritises a card-based vehicle browsing layout with prominent filters, a comparison tray, and a simplified multi-step checkout (destination → add-ons → KYC → payment). The admin command centre prioritises information density and glanceability: the fleet matrix uses colour coding consistent with the statuses defined in FR-10, the live map view is the default landing screen for fleet oversight, and the KYC verification queue and inspection checklist are presented as focused, single-task workspaces to reduce administrator error.

[INSERT DIAGRAM: Interface Wireframes / High-Fidelity Mockups]

Insert wireframes or high-fidelity mockups (from Figma/UX Pilot) for: (1) Customer vehicle search & comparison screen, (2) Booking & add-ons checkout flow, (3) Admin Fleet Matrix dashboard, (4) Admin live tracking map with geo-fence alert, (5) Digital inspection checklist screen.

5.7 System Process Design

The dynamic behaviour of key processes is modelled using sequence diagrams, showing the interaction between the client, API, database, and external services over time for the platform's

most critical workflows: booking with deposit placement, geo-fence breach alerting, and overdue penalty calculation.

[INSERT DIAGRAM: Sequence Diagram – Booking with Deposit Hold (UC-01)]

Insert a sequence diagram with lifelines Customer (React Client), Express API, MongoDB, Payment Gateway, showing: search/select vehicle → POST /bookings → validate availability → redirect to Payment Gateway → charge rental + place deposit hold → gateway webhook confirms → API creates Booking, Payment, and SecurityDeposit documents → Socket.io emits fleet-status update → notification sent to customer.

[INSERT DIAGRAM: Sequence Diagram – Overdue Penalty Calculation (Automation Engine)]

Insert a sequence diagram with lifelines Automation Engine (scheduled job), MongoDB, Notification Service, showing: hourly trigger → query bookings past endDate and grace period → compute penalty amount → update booking status to 'Overdue' and vehicle status matrix → send overdue notification via Twilio/SendGrid → log penalty for admin review.

5.8 Security Design

Security controls are layered across the application. Authentication uses JSON Web Tokens (JWT) with short-lived access tokens and refresh tokens; passwords are hashed using a strong adaptive hashing algorithm (e.g. bcrypt) and never stored in plaintext. Authorisation is enforced through role-based middleware in the Express API, ensuring administrative endpoints (fleet matrix, KYC verification, financial analytics) are inaccessible to customer accounts. KYC documents in the Document Vault are stored in access-controlled cloud storage with signed, time-limited URLs rather than public links, and sensitive fields (document numbers, payment references) are encrypted at rest.

All payment operations are delegated to PCI-DSS-compliant third-party gateways (M-Pesa/Flutterwave/Stripe) so that raw card or mobile money credentials never transit or persist on the application's own servers. API input is validated and sanitised server-side to mitigate injection attacks, and all administrative actions affecting deposits, KYC status, and inspection records are logged with a timestamp and actor identity to preserve an audit trail supporting the dispute-resolution needs identified in NFR-07.